

Генератор интеграций с платёжными провайдерами

Инструмент, который читает документацию платёжного провайдера и собирает из неё готовую интеграцию для Space Payments: сервис, инструкцию, тестовые примеры и отчёт о том, что осталось проверить человеку.

Сегодня показываем генерацию: одна команда, четыре файла.

В прошлый раз был только разбор без генерации. Сегодня из него собирается код.

Мы услышали и переписали приоритеты

- 1** **Инструмент решает больше сам.** Генерация теперь не блокируется ничем: везде лучшая догадка из словарей плюс TODO, а не пустое место.
- 2** **Оцениваете сгенерированный класс, не живой трафик.** Мок-провайдер ушёл в бонус, приоритет отдан генераторам.
- 3** **Проверять будете на других провайдерах.** Уже гоняем на настоящих публичных спеках, не только на NovaPay.
- 4** **Контракт BaseService несущественен.** Держим как допущение в данных, не тратим время на его уточнение.

Сегодня — прогон, а не рассказ о том, что мы поняли правильно.

2. Вот смотрите, как оно работает

Одна команда, четыре файла

```
$ ./integrate --spec spec/fixtures/specs/novapay.yaml --provider novapay
Генерация сервиса...      ok (379 строк) -> output/novapay_service.rb
Генерация документации... ok (218 строк) -> output/INTEGRATION.md
Генерация фикстур...      ok (461 строка) -> output/fixtures.json
Генерация отчёта...       ok (229 строк) -> output/report.md
Предупреждения: 18 (0 ошибок, 6 предупреждений, 12 справок)

$ ruby -c output/novapay_service.rb
Syntax OK
$ bundle exec rubocop --config config/rubocop_generated.yml output/novapay_service.rb
1 file inspected, no offenses detected
$ bundle exec rspec
771 examples, 0 failures
```

Вопрос к вам



Пять артефактов, ноль ручных шагов между ними — этого достаточно как «понятный последовательный процесс», или ждёте от нас чего-то ещё между командой и результатом?

output/novapay_service.rb, сверху вниз

- 1** `STATUS_MAP / ERROR_MAP / RETRY_POLICY / EVENT_MAP` — замороженные хеши, не ветки `if`.
- 2** `check_conditions` отклоняет сумму меньше 1000 ₽, в комментарии виден пересчёт `minimum: 100000` копеек.
- 3** **409 на создании — не ошибка.** `DEDUP_STATUS`, ответ читается как успех: взять существующую выплату, не отклонять.
- 4** `process_callback`: подпись → поиск по `payout_id`, потом по `external_id` → все четыре события → отказ на неизвестном.
- 5** `verify_signature!` — `OpenSSL::HMAC` плюс постоянное по времени сравнение, без заголовка — явный отказ.
- 6** `idempotency_key_for` — UUID v5 от `operation.id`, детерминирован.
- 7** `cancel` и `balance` — отдельные публичные методы с пометкой «вне контракта» прямо в комментарии.
- 8** **Секреты** — только `provider.credentials[...]`, ни одного литерала в коде.

Девять разделов, особенно раздел про допущения

- 1–2. Авторизация и секрет, ENV-переменные
- 3. Таблица методов с идемпотентностью
- 4–5. Статусы и события, ошибки с действиями и `RETRY_POLICY`
- 6. YAML-фрагмент `ProviderGateway` с тем же пересчётом суммы, что в сервисе
- 7–8. Схема подписи с примером без секрета, пошаговое подключение

Раздел 9. Принятые допущения

Восемь пунктов из `rules/assumptions.yml` плюс допущения именно этого прогона: две условных обязательности с уверенностью 0.50, ограничение отмены с уверенностью 0.60. Ничего не спрятано за красивым текстом — каждая догадка видна и подписана источником.

Вопрос к вам



Раздел 6, `ProviderGateway` -конфиг, мы придумали его форму сами, реального интерфейса нет. Такая штука вообще нужна платформе, или лишняя?

Настоящая подпись, покрытие одной цифрой, готовый Overlay

5 / 15 / 6

запросов / ответов /
уведомлений

fixtures.json, 461 строка

НМАС

подпись уведомления
настоящая

SHA-256(raw_body,
test_webhook_secret) — юнит-
тест пересчитывает и сверяет

76 %

покрытие спецификации

55 из 72 элементов, формула в
report.md

7

разделов отчёта

сводка, покрытие,
неоднозначности, вне
контракта, противоречия,
справки, чеклист

Готовый Overlay из раздела «Неоднозначности»

`card_number` выглядит условно обязательным при `type=card`, схема этого не задаёт — отчёт печатает готовый YAML-фрагмент `x-jsonschema-if / x-jsonschema-then`, который закрывает предупреждение.

Adyen Transfers — спека, которую мы не писали

```
$ ./integrate --spec spec/fixtures/specs/real/adyen_transfers_v4.yaml --provider adyen_transfers
Генерация сервиса...      ok (778 строк)  -> output/adyen_transfers_service.rb
Генерация документации... ok (354 строки) -> output/INTEGRATION.md
Генерация фикстур...      ok (2335 строк) -> output/fixtures.json
Генерация отчёта...       ok (862 строки) -> output/report.md
Предупреждения: 338 (0 ошибок, 122 предупреждения, 216 справок)

Покрытие: 34 % (202 из 588 элементов) — 99 схем, 437 полей

$ ruby -c output/adyen_transfers_service.rb
Syntax OK
```

Вопрос к вам



Низкое покрытие на чужой спеке — это честная цифра о размере Adyen, не провал инструмента. Мы тут не ошибаемся в интерпретации критерия универсальности?

Где мы сейчас, что доделываем к 6 сентября

Чего пока нет

- Мок-провайдера — по вашим словам, бонус
- `./integrate --all` со сводной таблицей
- Своих трёх альтернативных спек: карта, депозиты, дырявая
- `--fix` и сверки запросов со спекой

Что дальше, до третьего чекпоинта

- `--all` со сводной таблицей по своим и чужим спекам
- `broken.yaml` → отчёт → `--fix` → повторный прогон
- Сверка сформированных запросов со спекой
- **Веб-интерфейс:** тонкий Rack + ERB, без npm — прогон из браузера

Риск, который мы держим в голове: генераторы важнее веб-интерфейса, если придётся выбирать — выбираем генераторы.

Пять вопросов по убыванию цены ответа

- 1 Выражения платформы в `rules/contract.yml` — `operation.recipient_phone`, `recipient_type`, `bank_code`, `card_number`, `provider_operation_id` — это наше допущение. Так у вас и есть? И как сервис должен сохранить id провайдера после создания?

Влияет на весь payload сервиса, а не на один метод.
- 2 `process_callback` получает `{ body:, headers: }` или уже разобранный JSON? Если тело уже разобрано, откуда брать сырые байты для подписи?

Без ответа проверка подписи может быть в принципе невозможна на практике.
- 3 Блиц по канону, одной фразой на каждый: `accepted` — принято или зачислено; `cancelled` — навсегда `rejected` или будет четвёртый статус; `amount_limit_exceeded` — лимит операции или суточный; что делать на 402; в вебхуке при расхождении `event` и `status` что первично?
- 4 `ProviderGateway`-конфиг из раздела 6 — вам такое вообще нужно, или мы рисуем несуществующий интерфейс?
- 5 Что важнее к третьему чекпоинту: сводная таблица по чужим спекам или цикл `broken.yml` → отчёт → `--fix` → повторный прогон?

Если время останется: считается ли output/ в доле Ruby?